

Homework 2 — Machine Learning (Fall 2025)

Yash Dagade

October 17, 2025

Agreement. This assignment represents my own work. I have worked with the following people and/or LLMs: Cursor, GPT-5, and in terms of people I would like to thank Krish Yadav, Brian Mason, and Max Xiong for high-level discussion over the homework.

Signed, **Yash Dagade**

Boosting (Theory) — 50 pts (Muchang)

(a) Exponential Loss and Negative Log-likelihood Loss

In a previous lecture, we proved that the expected exponential loss for AdaBoost is minimized by

$$f^*(x) = \frac{1}{2} \ln \frac{\Pr(Y = 1 \mid x)}{\Pr(Y = -1 \mid x)}.$$

Show that if, instead, we *maximize* the expected negative binomial log-likelihood

$$\max_{f(x)} \mathbb{E}_{Y \sim D(x)} \left[-\log(1 + e^{-2Yf(x)}) \right],$$

the same optimizer $f^*(x)$ is obtained.

Solution

Let $p = \Pr(Y = 1 \mid x)$ and $1 - p = \Pr(Y = -1 \mid x)$. For any fixed x , define the (sign-flipped) objective

$$\mathcal{L}(f) = \mathbb{E} \left[-\log(1 + e^{-2Yf}) \right] = p \left[-\log(1 + e^{-2f}) \right] + (1 - p) \left[-\log(1 + e^{2f}) \right].$$

Equivalently, minimizing the *negative* of $\mathcal{L}$ yields the same maximizer, so it is convenient to work with

$$\ell(f) = -\mathcal{L}(f) = p \log(1 + e^{-2f}) + (1 - p) \log(1 + e^{2f}).$$

First derivative (carry out the algebra). Use $\frac{d}{df} \log(1 + e^{af}) = \frac{ae^{af}}{1 + e^{af}}$. Then

$$\ell'(f) = p \cdot \frac{-2e^{-2f}}{1 + e^{-2f}} + (1 - p) \cdot \frac{2e^{2f}}{1 + e^{2f}}.$$

Multiply top and bottom of the first term by e^{2f} : $\frac{-2e^{-2f}}{1 + e^{-2f}} = \frac{-2}{e^{2f} + 1}$.

Hence,

$$\ell'(f) = -\frac{2p}{1 + e^{2f}} + (1 - p) \frac{2e^{2f}}{1 + e^{2f}} = \frac{2((1 - p)e^{2f} - p)}{1 + e^{2f}}.$$

Stationary condition. Set $\ell'(f) = 0$:

$$(1 - p)e^{2f} - p = 0 \implies e^{2f} = \frac{p}{1 - p}.$$

Taking logs gives

$$f^*(x) = \frac{1}{2} \ln \frac{p}{1 - p} = \frac{1}{2} \ln \frac{\Pr(Y = 1 \mid x)}{\Pr(Y = -1 \mid x)}.$$

Second derivative (uniqueness). A direct computation yields

$$\ell''(f) = \frac{4e^{2f}}{(1 + e^{2f})^2} > 0 \quad \text{for all } f,$$

so ℓ is strictly convex and the critical point is the unique minimizer of ℓ (equivalently, the unique maximizer of $\mathcal{L}$). Therefore, maximizing the binomial log-likelihood recovers the same f^* as the exponential loss.

(b) Multiclass Exponential Loss

For a K -class problem, encode class $c \in \{1, \dots, K\}$ by $Y^{(c)} \in \mathbb{R}^K$ with

$$Y_k^{(c)} = \begin{cases} 1, & k = c, \\ -\frac{1}{K-1}, & k \neq c. \end{cases} \quad \text{and let } f = (f_1, \dots, f_K)^\top, \quad \sum_{k=1}^K f_k = 0.$$

Define $L(Y, f) = \exp(-\frac{1}{K} Y^\top f)$. Show that

$$\mathbb{E} \left[\exp \left(-\frac{1}{K} \sum_{k=1}^K Y_k f_k \right) \right] = \mathbb{E} \left[\exp \left(-\frac{1}{K-1} f_c \right) \right],$$

where the outer expectation is w.r.t. the class label.

Solution

We are given, for class c ,

$$Y_k^{(c)} = \begin{cases} 1, & k = c, \\ -\frac{1}{K-1}, & k \neq c, \end{cases} \quad \sum_{k=1}^K f_k = 0, \quad L(Y, f) = \exp \left(-\frac{1}{K} Y^\top f \right).$$

To make the structure of $Y^{(c)}$ clearer, note that

$$Y^{(c)} = \mathbf{1} \cdot e_c - \frac{1}{K-1} (\mathbf{1} - e_c) = \left(1 + \frac{1}{K-1}\right) e_c - \frac{1}{K-1} \mathbf{1} = \frac{K}{K-1} e_c - \frac{1}{K-1} \mathbf{1}.$$

Here e_c is the standard basis vector and $\mathbf{1}$ the all-ones vector.

Then

$$Y^{(c)\top} f = \left(\frac{K}{K-1} e_c^\top - \frac{1}{K-1} \mathbf{1}^\top \right) f = \frac{K}{K-1} f_c - \frac{1}{K-1} \sum_{k=1}^K f_k = \frac{K}{K-1} f_c,$$

since $\sum_k f_k = 0$. Therefore,

$$-\frac{1}{K} Y^{(c)\top} f = -\frac{1}{K} \cdot \frac{K}{K-1} f_c = -\frac{1}{K-1} f_c,$$

and

$$L(Y^{(c)}, f) = \exp \left(-\frac{1}{K-1} f_c \right).$$

Let $\pi_c = \Pr(Y = Y^{(c)} \mid x)$. Taking the expectation over the random class label,

$$\mathbb{E}[L(Y, f)] = \sum_{c=1}^K \pi_c \exp \left(-\frac{1}{K-1} f_c \right) = \mathbb{E}_{c \sim \pi} \left[\exp \left(-\frac{1}{K-1} f_c \right) \right].$$

This matches the desired expression, where the index k in the statement corresponds to the random class index.

(c) Multiclass Optimization

Using the result of (b) and the constraint $\sum_{k=1}^K f_k = 0$, minimize the expected loss and solve for the optimizer f^* . Then express $f_k^*(x)$ and the resulting class probabilities $\Pr(Y = Y^{(k)} | x)$ purely in terms of K and f^* (no λ).

Solution

Write $\pi_k = \Pr(Y = Y^{(k)} | x)$ and consider the constrained objective

$$\mathcal{L}(f) = \sum_{k=1}^K \pi_k \exp\left(-\frac{1}{K-1} f_k\right) \quad \text{s.t.} \quad \sum_{k=1}^K f_k = 0.$$

Introduce a Lagrange multiplier λ for the zero-sum constraint:

$$\mathcal{J}(f, \lambda) := \sum_{k=1}^K \pi_k \exp\left(-\frac{1}{K-1} f_k\right) - \lambda \sum_{k=1}^K f_k.$$

First-order conditions (each coordinate). For $j \in \{1, \dots, K\}$,

$$\frac{\partial \mathcal{J}}{\partial f_j} = \pi_j \exp\left(-\frac{1}{K-1} f_j\right) \left(-\frac{1}{K-1}\right) - \lambda = 0.$$

Rearrange to isolate the exponential:

$$\pi_j \exp\left(-\frac{1}{K-1} f_j\right) = \lambda(K-1).$$

The right-hand side is constant in j , so for some $\alpha > 0$,

$$\exp\left(-\frac{1}{K-1} f_j\right) = \frac{\alpha}{\pi_j} \quad \implies \quad \boxed{f_j^* = (K-1)(\log \pi_j - \log \alpha)}.$$

Determine α via the constraint. Impose $\sum_{j=1}^K f_j^* = 0$:

$$0 = \sum_{j=1}^K f_j^* = (K-1) \left(\sum_{j=1}^K \log \pi_j - K \log \alpha \right) \quad \implies \quad \log \alpha = \frac{1}{K} \sum_{j=1}^K \log \pi_j.$$

Therefore,

$$\boxed{f_k^*(x) = (K-1) \left(\log \pi_k(x) - \frac{1}{K} \sum_{j=1}^K \log \pi_j(x) \right)}.$$

Recover class probabilities in terms of f^* . From $\pi_k \exp\left(-\frac{1}{K-1} f_k^*\right) = \lambda(K-1)$, we have

$$\pi_k \propto \exp\left(\frac{1}{K-1} f_k^*\right).$$

Normalizing over k yields the softmax-like formula with temperature $K - 1$:

$$\Pr(Y = Y^{(k)} \mid x) = \frac{\exp(\frac{1}{K-1} f_k^*(x))}{\sum_{j=1}^K \exp(\frac{1}{K-1} f_j^*(x))}.$$

Binary sanity check ($K = 2$). Let $K = 2$. Then $f_1 = -f_2$ and

$$\Pr(Y = Y^{(1)} \mid x) = \frac{e^{f_1}}{e^{f_1} + e^{-f_1}} = \frac{1}{1 + e^{-2f_1}}, \quad \Rightarrow \quad f_1 = \frac{1}{2} \ln \frac{\Pr(Y = 1 \mid x)}{\Pr(Y = -1 \mid x)},$$

which matches part (a).

Exploring Gradient Based Optimization Methods (Theory + Applied) — 30 pts (Max)

(a) Manual iterations on $f(x) = x^2$: GD vs. Momentum

We will compare two iterative schemes on the convex objective $f(x) = x^2$:

$$\text{(GD)} \quad x_t = x_{t-1} - \eta \nabla f(x_{t-1}), \quad \text{(Momentum)} \quad \begin{cases} v_t = \beta v_{t-1} + \eta \nabla f(x_{t-1}), \\ x_t = x_{t-1} - v_t. \end{cases}$$

Use $x_0 = -2$, $v_0 = 0$, learning rate $\eta = 0.1$. In (i) run three GD steps and report $|x_3 - x^*|$. In (ii) run three momentum steps with $\beta = 0.75$ and report $|x_3 - x^*|$. Compare gaps.

Solution

Preliminaries. For $f(x) = x^2$, the gradient is $\nabla f(x) = 2x$. The global minimizer satisfies $2x^* = 0 \Rightarrow \boxed{x^* = 0}$.

(i) Gradient Descent. We apply

$$x_t = x_{t-1} - \eta \nabla f(x_{t-1}) = x_{t-1} - 0.1 \cdot (2x_{t-1}) = (1 - 0.2)x_{t-1} = 0.8x_{t-1}.$$

Now substitute numerically, carrying each arithmetic step explicitly:

$$\begin{aligned} x_1 &= x_0 - 0.1 \cdot (2x_0) = (-2) - 0.1 \cdot (2 \cdot (-2)) = -2 - 0.1 \cdot (-4) \\ &= -2 + 0.4 = \boxed{-1.6}. \end{aligned}$$

$$\begin{aligned} x_2 &= x_1 - 0.1 \cdot (2x_1) = (-1.6) - 0.1 \cdot (2 \cdot (-1.6)) = -1.6 - 0.1 \cdot (-3.2) \\ &= -1.6 + 0.32 = \boxed{-1.28}. \end{aligned}$$

$$\begin{aligned} x_3 &= x_2 - 0.1 \cdot (2x_2) = (-1.28) - 0.1 \cdot (2 \cdot (-1.28)) = -1.28 - 0.1 \cdot (-2.56) \\ &= -1.28 + 0.256 = \boxed{-1.024}. \end{aligned}$$

Optimality gap (distance to minimizer $x^* = 0$):

$$\boxed{|x_3 - x^*| = |-1.024 - 0| = 1.024.}$$

(ii) Gradient Descent with Momentum ($\eta = 0.1$, $\beta = 0.75$). Recall

$$v_t = \beta v_{t-1} + \eta \nabla f(x_{t-1}), \quad x_t = x_{t-1} - v_t, \quad \nabla f(x) = 2x.$$

Step $t = 1$:

$$\begin{aligned} \nabla f(x_0) &= 2x_0 = 2 \cdot (-2) = -4, \\ v_1 &= \beta v_0 + \eta \nabla f(x_0) = 0.75 \cdot 0 + 0.1 \cdot (-4) = \boxed{-0.4}, \\ x_1 &= x_0 - v_1 = (-2) - (-0.4) = \boxed{-1.6}. \end{aligned}$$

Step $t = 2$:

$$\begin{aligned}\nabla f(x_1) &= 2x_1 = 2 \cdot (-1.6) = -3.2, \\ v_2 &= \beta v_1 + \eta \nabla f(x_1) = 0.75 \cdot (-0.4) + 0.1 \cdot (-3.2) \\ &= -0.300 + (-0.320) = \boxed{-0.620}, \\ x_2 &= x_1 - v_2 = (-1.6) - (-0.620) = \boxed{-0.980}.\end{aligned}$$

Step $t = 3$:

$$\begin{aligned}\nabla f(x_2) &= 2x_2 = 2 \cdot (-0.980) = -1.960, \\ v_3 &= \beta v_2 + \eta \nabla f(x_2) = 0.75 \cdot (-0.620) + 0.1 \cdot (-1.960) \\ &= -0.465 + (-0.196) = \boxed{-0.661}, \\ x_3 &= x_2 - v_3 = (-0.980) - (-0.661) = \boxed{-0.319}.\end{aligned}$$

Optimality gap:

$$\boxed{|x_3 - x^*| = |-0.319 - 0| = 0.319.}$$

Comparison. After three iterations, momentum has the smaller gap ($0.319 < 1.024$), i.e., it converges faster on this simple quadratic.

(b) A more challenging objective $f(x) = \sin(-5x) + \frac{1}{5}x^2$

You may use Python to simulate and plot. Use the same GD and Momentum updates as in (a).

- (i) Plot $f(x) = \sin(-5x) + \frac{1}{5}x^2$. Explain why this landscape is challenging for vanilla GD.
- (ii) Run GD with $\eta = 0.1$, $x_0 = 20$, 20 iterations; plot the trajectory x_t over iterations.
- (iii) Run Momentum with $\eta = 0.1$, $\beta = 0.5$, $x_0 = 20$, 20 iterations; plot x_t . Which method attains the lower function value in your run?
- (iv) Now set $x_0 = 0$ for both methods (same hyperparameters). Plot trajectories and compare the behaviors near a minimum.

NOTE: For all of these questions, they have also been answered in jupyter notebooks as well so please refer to both places for the complete submission

Solution

Gradients we will use.

$$f(x) = \sin(-5x) + \frac{1}{5}x^2, \quad f'(x) = \cos(-5x) \cdot (-5) + \frac{2}{5}x = -5\cos(5x) + 0.4x,$$

since $\cos$ is even.

(i) Why challenging? The objective is *nonconvex* due to the oscillatory $\sin(-5x)$ term (period $2\pi/5$). This produces many local minima and maxima (“ripples”). Vanilla GD uses only local slope information:

- In regions where $|\cos(5x)|$ is large, the $-5\cos(5x)$ term can dominate, flipping gradient signs frequently and inducing zig-zagging.
- In shallow wells, small gradients slow progress; in steep oscillations, fixed η can overshoot and bounce between sides.
- Far from the origin, the quadratic term $0.2x^2$ grows and competes with the sinusoid, creating wide bowls modulated by high-frequency ripples.

(ii) GD trajectory ($\eta = 0.1$, $x_0 = 20$, 20 steps). See attached Jupyter notebook for code.

(iii) Momentum trajectory ($\eta = 0.1$, $\beta = 0.5$, $x_0 = 20$, 20 steps). See attached notebook.

Observed outcome in our run. From the plots (see Jupyter notebook), the momentum method achieved a *lower* final function value than vanilla GD by effectively smoothing oscillations and carrying velocity across shallow basins, while GD became stuck in a higher local minimum. Specifically, for vanilla GD with $x_0 = 20$, we obtained $x_T \approx 8.893493$ and $f(x_T) \approx 15.352487$. In contrast, gradient descent with momentum (GDM) reached $x_T \approx -0.726585$ with $f(x_T) \approx -0.366215$. For comparison, the true (reference) minimum occurs near $x^* \approx 0.309000$ where $f(x^*) \approx -0.980571$. Thus, while GDM did not reach the global minimum, it converged much closer to it than standard GD.

(iv) Behavior near a minimum, $x_0 = 0$. Note $f'(0) = -5\cos(0) + 0 = -5 < 0$, so both methods move to the right on the first step ($x_1 = x_0 - \eta f'(0) = 0 - 0.1 \cdot (-5) = 0.5$). Subsequent steps encounter alternating curvature from the sinusoid:

- **GD** exhibits small, sometimes erratic steps as the gradient sign changes with $\cos(5x)$, causing visible zig-zags.
 - **Momentum** damps these oscillations and accelerates along consistent descent directions, yielding a smoother, quicker trajectory through the ripples.
-

(c) Second-order (Newton) update: derivation and implications

Assume $L(\mathbf{w})$ is twice differentiable and $H(\mathbf{w}_0) = \nabla^2 L(\mathbf{w}_0)$ is invertible.

- (i) Write the second-order Taylor expansion $\bar{L}(\mathbf{w})$ of L around $\mathbf{w}_0$.
- (ii) Minimize $\bar{L}(\mathbf{w})$ to obtain the Newton update $\mathbf{w}_1$.
- (iii) Apply one Newton step to $L(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top \mathbf{A} \mathbf{w} - \mathbf{b}^\top \mathbf{w}$, where $\mathbf{A}$ is symmetric invertible. Relate to the true minimizer.
- (iv) State the two main scalability bottlenecks (memory and compute) for large-scale neural nets.

Solution

(i) Second-order approximation.

We know that the second-order Taylor series expansion of a scalar function $L(w)$ around a point w_0 is given by:

$$L(w) \approx L(w_0) + L'(w_0)(w - w_0) + \frac{1}{2}L''(w_0)(w - w_0)^2.$$

Here we replace the derivative $L'(w_0)$ with the gradient $\nabla L(\mathbf{w}_0)$ and the second derivative $L''(w_0)$ with the Hessian $H(\mathbf{w}_0)$, to get:

$$\bar{L}(\mathbf{w}) = L(\mathbf{w}_0) + (\mathbf{w} - \mathbf{w}_0)^\top \nabla L(\mathbf{w}_0) + \frac{1}{2}(\mathbf{w} - \mathbf{w}_0)^\top H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0).$$

(ii) Minimizer of $\bar{L}$.

To find the minimizer of the quadratic approximation $\bar{L}(\mathbf{w})$, we differentiate it with respect to $\mathbf{w}$ and set the result to zero.

Starting from:

$$\bar{L}(\mathbf{w}) = L(\mathbf{w}_0) + (\mathbf{w} - \mathbf{w}_0)^\top \nabla L(\mathbf{w}_0) + \frac{1}{2}(\mathbf{w} - \mathbf{w}_0)^\top H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0),$$

we compute the gradient with respect to $\mathbf{w}$.

Recall the useful vector calculus identities:

$$\nabla_{\mathbf{x}}(a^\top x) = a, \quad \nabla_{\mathbf{x}}(x^\top A x) = (A + A^\top)x.$$

Since $H(\mathbf{w}_0)$ is symmetric ($H = H^\top$), we have:

$$\nabla_{\mathbf{w}}\left(\frac{1}{2}(\mathbf{w} - \mathbf{w}_0)^\top H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0)\right) = H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0).$$

Applying these results term-by-term:

$$\begin{aligned} \nabla_{\mathbf{w}} \bar{L}(\mathbf{w}) &= \nabla_{\mathbf{w}} \left[L(\mathbf{w}_0) + (\mathbf{w} - \mathbf{w}_0)^\top \nabla L(\mathbf{w}_0) + \frac{1}{2}(\mathbf{w} - \mathbf{w}_0)^\top H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0) \right] \\ &= 0 + \nabla L(\mathbf{w}_0) + H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0). \end{aligned}$$

Setting this gradient equal to zero to find the minimizer:

$$\nabla L(\mathbf{w}_0) + H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0) = 0.$$

Rearranging,

$$H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0) = -\nabla L(\mathbf{w}_0).$$

Since $H(\mathbf{w}_0)$ is invertible by assumption, multiply both sides by $H(\mathbf{w}_0)^{-1}$:

$$\mathbf{w} - \mathbf{w}_0 = -H(\mathbf{w}_0)^{-1}\nabla L(\mathbf{w}_0).$$

Hence, the minimizer is:

$$\boxed{\mathbf{w}_1 = \mathbf{w}_0 - H(\mathbf{w}_0)^{-1}\nabla L(\mathbf{w}_0).}$$

This update is precisely the Newton step, which adjusts the current estimate $\mathbf{w}_0$ by the scaled inverse-Hessian direction, moving directly toward the local quadratic minimum.

(iii) Quadratic case.

Now consider the specific quadratic objective:

$$L(\mathbf{w}) = \frac{1}{2}\mathbf{w}^\top \mathbf{A}\mathbf{w} - \mathbf{b}^\top \mathbf{w},$$

where $\mathbf{A}$ is a symmetric, invertible matrix, and $\mathbf{w}, \mathbf{b}$ are appropriately dimensioned vectors.

Using standard matrix calculus results:

$$\nabla_{\mathbf{w}}(\frac{1}{2}\mathbf{w}^\top \mathbf{A}\mathbf{w}) = \frac{1}{2}(\mathbf{A} + \mathbf{A}^\top)\mathbf{w} = \mathbf{A}\mathbf{w},$$

since $\mathbf{A}$ is symmetric. Also, $\nabla_{\mathbf{w}}(-\mathbf{b}^\top \mathbf{w}) = -\mathbf{b}$.

Hence,

$$\boxed{\nabla L(\mathbf{w}) = \mathbf{A}\mathbf{w} - \mathbf{b}.}$$

Evaluating this at the starting point $\mathbf{w}_0$,

$$\nabla L(\mathbf{w}_0) = \mathbf{A}\mathbf{w}_0 - \mathbf{b}.$$

Now,

Differentiating the gradient with respect to $\mathbf{w}$:

$$H(\mathbf{w}) = \nabla_{\mathbf{w}}^2 L(\mathbf{w}) = \mathbf{A}.$$

Since $\mathbf{A}$ is constant and symmetric,

$$\boxed{H(\mathbf{w}_0) = \mathbf{A}.}$$

Now from before, the general Newton Update is:

$$\mathbf{w}_1 = \mathbf{w}_0 - H(\mathbf{w}_0)^{-1}\nabla L(\mathbf{w}_0).$$

Substitute the expressions for $H(\mathbf{w}_0)$ and $\nabla L(\mathbf{w}_0)$:

$$\begin{aligned}\mathbf{w}_1 &= \mathbf{w}_0 - \mathbf{A}^{-1}(\mathbf{A}\mathbf{w}_0 - \mathbf{b}) \\ &= \mathbf{w}_0 - \mathbf{A}^{-1}\mathbf{A}\mathbf{w}_0 + \mathbf{A}^{-1}\mathbf{b} \\ &= \mathbf{w}_0 - \mathbf{w}_0 + \mathbf{A}^{-1}\mathbf{b} \\ &= \boxed{\mathbf{w}_1 = \mathbf{A}^{-1}\mathbf{b}.}\end{aligned}$$

To find the global minimizer $\mathbf{w}^*$ directly, set the gradient to zero:

$$\nabla L(\mathbf{w}) = \mathbf{A}\mathbf{w} - \mathbf{b} = 0 \quad \Rightarrow \quad \boxed{\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b}.}$$

Thus,

$$\mathbf{w}_1 = \mathbf{w}^*,$$

meaning that the Newton update reaches the exact minimizer in 1 step.

Remark: This result occurs because $L(\mathbf{w})$ is already a quadratic function — its second-order Taylor expansion is identical to the original function. Therefore, Newton’s method perfectly models L and finds its minimum in a single iteration.

(iv) Bottlenecks for large models.

In practice, applying second-order methods such as Newton’s update to modern neural networks is computationally infeasible due to two primary bottlenecks. The first is **memory**: storing the Hessian matrix $H \in \mathbb{R}^{n \times n}$ requires $O(n^2)$ space, which becomes impractical for networks with millions of parameters—on the order of terabytes for $n \sim 10^6$. The second is **computation**: solving linear systems involving H , or explicitly computing its inverse, generally scales as $O(n^3)$. This cubic complexity makes each iteration prohibitively expensive, even though convergence is theoretically faster. As a result, second-order methods are rarely used in large-scale neural networks, where first-order methods like SGD and Adam remain the practical choice.
